

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

**FACULTAD DE INGENIERÍA ELÉCTRICA,
ELECTRÓNICA, INFORMÁTICA Y MECÁNICA**

**ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA
Y DE SISTEMAS**



INFORME TÉCNICO DE INVESTIGACIÓN

**“RUTAS MÍNIMAS CON DISTINTAS COLAS DE PRIORIDAD EN REDES
VIALES URBANAS DE LA CIUDAD DEL CUSCO”**

Asignatura: ALGORITMOS AVANZADOS (Semestre 2026-2)

Docente Responsable: Ing. Héctor Eduardo Ugarte Rojas

Identificador: GRUPO 5

Presentado por:

- | | |
|------------------------------------|--------|
| • Choquenaira Quispe, Noe Franklin | 133962 |
| • Porroa Sivana, Yeni Ruth | 120893 |
| • Quispe Rimachi, Romario | 164257 |
| • Yaranga Achahui, Aldo | 103179 |

Índice

1. Introducción	1
1.1. Contexto y Motivación	1
1.2. Planteamiento del Problema	1
1.3. Objetivos del Proyecto	2
1.3.1. Objetivo General	2
1.3.2. Objetivos Específicos	2
1.4. Alcance y Delimitación	2
1.5. Organización del Documento	3
2. Formulación Matemática del Problema y Caso de Estudio en Cusco	4
2.1. Definición Formal del Grafo Vial Urbano	4
2.2. Parámetros de Entrada, Variables de Decisión y Salidas del Sistema	4
2.2.1. Parámetros de Entrada	4
2.2.2. Variables de Decisión Binarias	4
2.2.3. Salidas del Sistema	5
2.3. Restricciones de Conservación de Flujo y No Negatividad	5
2.4. Función Objetivo y Modelo de Costo Generalizado para Cusco	5
2.4.1. Formulación de la Impedancia Vial Andina	5
2.5. Instancia Pequeña Resuelta Manualmente: Centro Histórico del Cusco	6
2.6. Traza Manual Paso a Paso del Algoritmo de Dijkstra ($k = 0$ a $k = 6$)	7
2.6.1. Reconstrucción de Rutas Mínimas Óptimas	8
3. Diseño Algorítmico y Estructuras de Colas de Prioridad	9
3.1. Algoritmo de Dijkstra con Cola de Prioridad Abstracta	9
3.2. Invariantes de Bucle y Prueba de Corrección	9
3.3. Estructuras de Datos de Cola de Prioridad Evaluadas	10
3.3.1. Min-Heap Binario Indexado (IndexedMinHeap)	10
3.3.2. 4-ary Heap Contiguo (DaryHeap)	11
3.3.3. Fibonacci Heap (FibonacciHeap)	11
3.3.4. Pairing Heap (Perspectiva Teórica)	11
3.4. Matriz Comparativa de Complejidad Asintótica	12
4. Diseño e Implementación del Sistema	13
4.1. Arquitectura Modular del Software	13
4.2. Descripción de Módulos del Sistema	13
4.3. Formatos de Datos y Serialización JSON	14
4.4. Contratos de Interfaz de las Estructuras de Datos	14
4.5. Implementación de Algoritmos y Estructuras Nucleares	15
Referencias Bibliográficas	19

Índice de figuras

1. Grafo vial dirigido representativo del Centro Histórico del Cusco ($|V| = 6, |E| = 9$) . . . 6
2. Diagrama arquitectural desacoplado del sistema en cuatro capas operativas 13

Índice de tablas

1. Tramos viales dirigidos, distancias métricas y costo de impedancia en el Cusco. 7
2. Traza matricial de distancias mínimas y predecesores en cada iteración del algoritmo. . . 8
3. Matriz comparativa de complejidades temporales y espaciales de colas de prioridad en Dijkstra 12

1. Introducción

1.1. Contexto y Motivación

La determinación eficiente de rutas mínimas en redes viales urbanas representa uno de los problemas fundamentales y de mayor impacto práctico en la ciencia de la computación, la ingeniería de transporte y la logística de atención a emergencias. En entornos metropolitanos contemporáneos, los sistemas de despacho para ambulancias, unidades de bomberos y patrullaje policial requieren calcular trayectorias óptimas en fracciones de segundo para minimizar los tiempos de respuesta, donde reducciones marginales de tiempo se traducen directamente en la preservación de vidas humanas.

La ciudad del Cusco, ubicada en la sierra sur del Perú a una altitud media de 3,399 m s. n. m., presenta una morfología urbana singular. Su Centro Histórico, declarado Patrimonio de la Humanidad por la UNESCO en 1983, superpone una traza urbana incaica y colonial caracterizada por calzadas sumamente angostas, pasajes adoquinados, alta densidad de giros obligatorios y severas pendientes topográficas que oscilan con frecuencia entre el 5% y más del 15%. En este contexto geográfico, la distancia euclidiana o la longitud puramente física de un tramo vial resultan insuficientes como métrica de optimización. Un trayecto físicamente más corto pero con un gradiente positivo pronunciado impone un esfuerzo motriz severo sobre un vehículo pesado de emergencia, reduciendo drásticamente su velocidad operativa y aumentando el riesgo de demoras críticas. Por ende, resulta imperativo formular modelos de costo generalizado que integren la topografía andina y evaluar algoritmos de caminos mínimos capaces de procesar mallas viales a gran escala con alta eficiencia temporal y espacial.

1.2. Planteamiento del Problema

El Algoritmo de Dijkstra (Dijkstra, 1959) es el estándar canónico para resolver el problema de caminos mínimos desde un origen único (*Single-Source Shortest Path*, SSSP) en grafos dirigidos con pesos no negativos ($w(e) \geq 0$). La complejidad asintótica de este algoritmo depende intrínsecamente de la estructura de datos utilizada para implementar la cola de prioridad que gestiona los vértices tentativos:

$$T(|V|, |E|) = O(|V| \cdot T_{\text{Extract-Min}} + |E| \cdot T_{\text{Decrease-Key}} + |V| \cdot T_{\text{Insert}}) \quad (1)$$

En la literatura teórica clásica, el Fibonacci Heap propuesto por Fredman y Tarjan (1987) ostenta la cota asintótica óptima conocida de $O(|E| + |V| \log |V|)$, al ejecutar la operación de disminución de clave (*Decrease-Key*) en tiempo amortizado $O(1)$ y la extracción del mínimo (*Extract-Min*) en $O(\log |V|)$. No obstante, estudios computacionales de referencia (Cherkassky et al., 1996; LaMarca & Ladner, 1996) han reportado que, en redes dispersas donde $|E| = O(|V|)$ (como las redes viales donde el grado promedio por nodo rara vez supera 4), la constante oculta en la notación asintótica del Fibonacci Heap, originada por la alocaión dinámica en memoria, la manipulación de múltiples punteros y los fallos en las líneas de caché (*cache misses*), degrada su rendimiento en tiempo real frente a montículos contiguos como el Binary Heap o el d -ary Heap.

El problema central que aborda este proyecto radica en investigar, implementar y contrastar cuantitativamente el comportamiento empírico y teórico de distintas colas de prioridad acopladas al algoritmo de Dijkstra, evaluando su desempeño real sobre la red vial urbana del Cusco bajo una función

de costo generalizado con impedancia topográfica.

1.3. Objetivos del Proyecto

1.3.1. Objetivo General

Diseñar, implementar y evaluar experimentalmente una solución computacional modular para el cálculo de rutas mínimas en la red vial urbana de la ciudad del Cusco, contrastando el desempeño en tiempo de ejecución, operaciones elementales y consumo de memoria del Algoritmo de Dijkstra instrumentado con tres estructuras de colas de prioridad: Min-Heap Binario Indexado, d -ary Heap y Fibonacci Heap.

1.3.2. Objetivos Específicos

1. **OE1 (Modelado Matemático y Topográfico):** Formular el grafo vial dirigido del Centro Histórico del Cusco y definir una función de costo generalizado que integre longitud física, velocidad de diseño por jerarquía vial y penalización por gradiente altitudinal andino.
2. **OE2 (Diseño e Implementación Algorítmica):** Implementar en Python 3 de manera pura y desacoplada las estructuras de datos de Min-Heap Binario Indexado, 4-ary Heap y Fibonacci Heap con soporte formal para la operación Decrease-Key en tiempo sublineal o constante amortizado.
3. **OE3 (Verificación y Pruebas Rigurosas):** Validar analíticamente una instancia pequeña de la red vial mediante traza manual paso a paso y construir una suite de pruebas automatizadas con Pytest bajo el estándar Arrange-Act-Assert (AAA) cubriendo casos básicos, límites, adversos y de escala.
4. **OE4 (Investigación y Aprendizaje Autónomo):** Fortalecer la autonomía técnica y metodológica mediante el diagnóstico riguroso de brechas conceptuales, la selección justificada del stack tecnológico y el diseño de un marco experimental reproducible.

1.4. Alcance y Delimitación

El proyecto se delimita formalmente en las siguientes dimensiones:

- **Delimitación Espacial:** La zona de estudio experimental inicial corresponde a la malla vial nuclear del Centro Histórico del Cusco (intersecciones entre Plaza Mayor, Plaza Regocijo, Qorikancha, San Francisco, San Pedro y Limacpampa), extensible a la red metropolitana completa mediante datos vectoriales de OpenStreetMap vía OSMnx.
- **Delimitación Algorítmica:** Se evalúa el algoritmo de Dijkstra en grafos estáticos dirigidos con pesos deterministas no negativos. No se contemplan pesos negativos ni grafos con propiedades estocásticas o dinámicas en tiempo real en este hito.
- **Alcance del Sistema:** Comprende la formulación matemática integral, el ejemplo manual resuelto, la arquitectura del prototipo inicial ejecutable, la suite de pruebas unitarias y los planes de investigación y experimentación.

1.5. Organización del Documento

El presente informe se estructura rigurosamente en las siguientes secciones temáticas: La Sección ?? expone los fundamentos teóricos y el diagnóstico de retos técnicos. La Sección ?? evalúa y justifica el stack tecnológico adoptado mediante una matriz multicriterio. La Sección 2 presenta la formulación matemática y el modelo de costo cusqueño. La Sección 3 detalla el diseño algorítmico, pseudocódigos, invariantes y complejidades. La Sección 4 describe la arquitectura del software y módulos implementados. La Sección ?? especifica la estrategia de pruebas unitarias (AAA). La Sección ?? formula las hipótesis y el diseño del banco de pruebas experimental. La Sección ?? documenta los resultados computacionales del prototipo inicial, la verificación analítica sobre Cusco y el protocolo de demostración en el sistema. Finalmente, las referencias en formato APA 7ma edición y los Anexos técnicos (manuales de instalación y ejecución, bitácora de aprendizaje y matriz de trazabilidad) complementan el informe.

2. Formulación Matemática del Problema y Caso de Estudio en Cusco

2.1. Definición Formal del Grafo Vial Urbano

El problema de cálculo de rutas mínimas en la red vial urbana del Cusco se formula formalmente como la determinación del camino de costo acumulado mínimo sobre un grafo dirigido y ponderado:

$$G = (V, E, W) \quad (2)$$

donde:

- $V = \{v_1, v_2, \dots, v_n\}$ es el conjunto finito de vértices que representan las intersecciones viales, nodos semaforizados o cambios de geometría del Centro Histórico.
- $E \subseteq V \times V$ es el conjunto de aristas dirigidas (arcos), donde cada par ordenado $e = (u, v) \in E$ representa un tramo de vía transitable exclusivamente en el sentido permitido desde la intersección u hacia la intersección v .
- $W : E \rightarrow \mathbb{R}^+$ es la función de costo aplicado o impedancia generalizada de viaje asignada a cada arista $e \in E$, cumpliendo estrictamente con la restricción de no negatividad:

$$w(u, v) \geq 0, \quad \forall (u, v) \in E \quad (3)$$

2.2. Parámetros de Entrada, Variables de Decisión y Salidas del Sistema

2.2.1. Parámetros de Entrada

1. El grafo vial estructurado $G = (V, E, W)$.
2. El nodo de origen $s \in V$, correspondiente a la estación o ubicación inicial de la unidad de emergencia (para el caso de estudio, $s = A$, Plaza Mayor del Cusco).
3. Los atributos físicos de cada arista $(u, v) \in E$: longitud métrica $L(u, v)$, tipo de vía $\text{tipo}(u, v)$ (avenida, calle céntrica, calle residencial) y la pendiente topográfica longitudinal $\text{pendiente}(u, v)$ expresada en porcentaje decimal.

2.2.2. Variables de Decisión Binarias

Para formular el problema bajo el enfoque clásico de programación entera en redes, se define para cada arista $(u, v) \in E$ una variable de flujo binaria:

$$x_{uv} = \begin{cases} 1, & \text{si el tramo vial } (u, v) \text{ forma parte del camino mínimo óptimo;} \\ 0, & \text{en caso contrario.} \end{cases} \quad (4)$$

2.2.3. Salidas del Sistema

1. Un vector de distancias mínimas definitivas $d : V \rightarrow \mathbb{R}^+$, donde $d[v]$ representa el costo mínimo de desplazamiento desde s hasta v .
2. Un vector de predecesores $\pi : V \rightarrow V \cup \{\text{NIL}\}$, que permite reconstruir de forma invertida la secuencia única de intersecciones que conforman la ruta mínima:

$$P(s, t) = \langle s = v_0, v_1, v_2, \dots, v_k = t \rangle \quad (5)$$

2.3. Restricciones de Conservación de Flujo y No Negatividad

El flujo neto en cada vértice $u \in V$ debe satisfacer el principio de balance de conservación de flujo unitario para un destino específico $t \in V \setminus \{s\}$:

$$\sum_{v:(u,v) \in E} x_{uv} - \sum_{v:(v,u) \in E} x_{vu} = \begin{cases} 1, & \text{si } u = s \text{ (fuente);} \\ -1, & \text{si } u = t \text{ (sumidero);} \\ 0, & \text{si } u \in V \setminus \{s, t\} \text{ (nodos de tránsito intermedio).} \end{cases} \quad (6)$$

Asimismo, se garantiza la ausencia de costos negativos:

$$x_{uv} \in \{0, 1\}, \quad w(u, v) \geq 0, \quad \forall (u, v) \in E \quad (7)$$

2.4. Función Objetivo y Modelo de Costo Generalizado para Cusco

La función objetivo busca minimizar el costo total acumulado del trayecto:

$$\text{mín} Z = \sum_{(u,v) \in E} w(u, v) \cdot x_{uv} \quad (8)$$

2.4.1. Formulación de la Impedancia Vial Andina

Para modelar de forma realista el comportamiento de vehículos de emergencia en Cusco, resulta indispensable formular una función de costo que no se limite a la distancia métrica. El costo aplicado $w(u, v)$ en segundos de viaje efectivo se modela como:

$$w(u, v) = T_{\text{base}}(u, v) \cdot \Phi(\text{pendiente}(u, v)) \cdot \Psi_{\text{vial}}(u, v) \quad (9)$$

donde:

- $T_{\text{base}}(u, v)$ es el tiempo de viaje base en flujo libre:

$$T_{\text{base}}(u, v) = \frac{L(u, v)}{V_{\text{nom}}(\text{tipo}(u, v))} \quad (10)$$

con velocidades nominales de diseño: $V_{\text{nom}}(\text{avenida}) = 11.11 \text{ m/s}$ (40 km/h), $V_{\text{nom}}(\text{calle_centro}) = 6.94 \text{ m/s}$ (25 km/h), y $V_{\text{nom}}(\text{calle_residencial}) = 5.56 \text{ m/s}$ (20 km/h).

- $\Phi(\text{pendiente})$ es el factor de penalización topográfica por gradiente positivo:

$$\Phi(\text{pendiente}) = 1.0 + \alpha \cdot \text{máx}(0, \text{pendiente}(u, v)) \quad (11)$$

donde el coeficiente $\alpha = 2.5$ calibra el esfuerzo motriz de vehículos pesados (ambulancias/camiones cisterna) al ascender pendientes andinas pronunciadas. Las pendientes descendentes no generan penalización motriz ($\text{máx}(0, \text{pendiente}) = 0$).

- Ψ_{vial} es el coeficiente de restricción de maniobrabilidad: $\Psi = 1.20$ para calles estrechas coloniales con adoquín y $\Psi = 1.00$ para avenidas amplias asfaltadas.

2.5. Instancia Pequeña Resuelta Manualmente: Centro Histórico del Cusco

Se define una instancia canónica de 6 intersecciones estratégicas ($|V| = 6$) y 9 tramos viales dirigidos ($|E| = 9$) representativa de la red vial cusqueña (Figura 1).

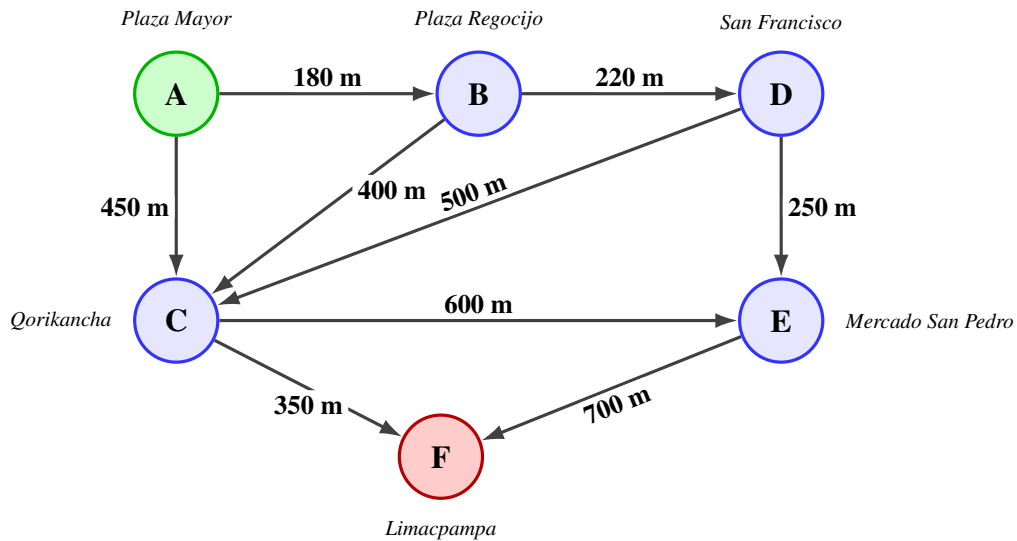


Figura 1

Grafo vial dirigido representativo del Centro Histórico del Cusco ($|V| = 6, |E| = 9$)

La Tabla 1 resume las características geométricas y físicas reales de cada tramo vial, exportada directamente desde el modelo computacional.

Tabla 1

Tramos viales dirigidos, distancias métricas y costo de impedancia en el Cusco.

Origen	Destino	Vía Urbana Asociada	Distancia (m)	Pendiente (%)	Tiempo Est. (s)
A	B	Calle Espaderos / Portal Carnicerías	180	+1.5	44.8
A	C	Calle Loreto / Pampa del Castillo	450	-2.0	108.0
B	D	Calle Garcilaso / Calle Marqués	220	+3.0	56.8
B	C	Calle San Bernardo / Almagro	400	-1.0	96.0
C	F	Calle Ahuacpinta / Limacpampa Grande	350	+0.5	36.5
C	E	Calle Matará / Tres Cruces de Oro	600	+2.0	151.2
D	E	Calle Santa Clara / Arco Santa Clara	250	+2.5	63.8
D	C	Calle Marqués / Mantas	500	-2.5	120.0
E	F	Calle Nueva Baja / Av. Tullumayo	700	-1.0	72.0

Nota. Tiempo de viaje estimado según la función de costo e impedancia por pendiente andina ($\alpha = 2.5$).

2.6. Traza Manual Paso a Paso del Algoritmo de Dijkstra ($k = 0$ a $k = 6$)

A continuación, se desarrolla analíticamente la traza manual paso a paso del Algoritmo de Dijkstra desde el origen $s = A$ (Plaza Mayor) utilizando la métrica de distancia física para verificación determinista:

■ Inicialización ($k = 0$):

$$d[A] = 0, \quad d[B] = \infty, \quad d[C] = \infty, \quad d[D] = \infty, \quad d[E] = \infty, \quad d[F] = \infty$$

$$\pi[\text{nodo}] = \text{NIL}, \quad \forall \text{nodo} \in V; \quad S = \emptyset; \quad Q = \{(A, 0), (B, \infty), (C, \infty), (D, \infty), (E, \infty), (F, \infty)\}$$

■ Paso 1 ($k = 1$): Se extrae el mínimo de Q : $u = A$ con $d[A] = 0$. Vértice permanente: $S = \{A\}$. Aristas salientes de A: (A, B) y (A, C) .

- Relajación de (A, B) : $d[A] + w(A, B) = 0 + 180 = 180 < d[B](\infty)$. Se actualiza $d[B] = 180$, $\pi[B] = A$. Decrease-Key($Q, B, 180$).

- Relajación de (A, C) : $d[A] + w(A, C) = 0 + 450 = 450 < d[C](\infty)$. Se actualiza $d[C] = 450$, $\pi[C] = A$. Decrease-Key($Q, C, 450$).

■ Paso 2 ($k = 2$): Se extrae el mínimo de Q : $u = B$ con $d[B] = 180$. Vértice permanente: $S = \{A, B\}$. Aristas salientes de B: (B, D) y (B, C) .

- Relajación de (B, D) : $d[B] + w(B, D) = 180 + 220 = 400 < d[D](\infty)$. Se actualiza $d[D] = 400$, $\pi[D] = B$. Decrease-Key($Q, D, 400$).

- Relajación de (B, C) : $d[B] + w(B, C) = 180 + 400 = 580 \not< d[C](450)$. No hay cambio.

■ Paso 3 ($k = 3$): Mínimos candidatos en Q : D ($d = 400$) y C ($d = 450$). Se extrae $u = D$ con $d[D] = 400$. $S = \{A, B, D\}$. Aristas salientes de D: (D, E) y (D, C) .

- Relajación de (D, E) : $d[D] + w(D, E) = 400 + 250 = 650 < d[E](\infty)$. Se actualiza $d[E] = 650$, $\pi[E] = D$. Decrease-Key($Q, E, 650$).

- Relajación de (D, C) : $d[D] + w(D, C) = 400 + 500 = 900 \not\leq d[C](450)$. No hay cambio.
- **Paso 4** ($k = 4$): Se extrae $u = C$ con $d[C] = 450$. $S = \{A, B, D, C\}$. Aristas salientes de C : (C, F) y (C, E) .
 - Relajación de (C, F) : $d[C] + w(C, F) = 450 + 350 = 800 < d[F](\infty)$. Se actualiza $d[F] = 800$, $\pi[F] = C$. Decrease-Key($Q, F, 800$).
 - Relajación de (C, E) : $d[C] + w(C, E) = 450 + 600 = 1050 \not\leq d[E](650)$. No hay cambio.
- **Paso 5** ($k = 5$): Se extrae $u = E$ con $d[E] = 650$. $S = \{A, B, D, C, E\}$. Arista saliente de E : (E, F) .
 - Relajación de (E, F) : $d[E] + w(E, F) = 650 + 700 = 1350 \not\leq d[F](800)$. No hay cambio.
- **Paso 6** ($k = 6$): Se extrae el último nodo restante $u = F$ con $d[F] = 800$. $S = V$. La cola de prioridad Q queda vacía. Finaliza el algoritmo.

La Tabla 2 sintetiza la evolución algorítmica iteración por iteración generada automáticamente por el solver.

Tabla 2

Traza matricial de distancias mínimas y predecesores en cada iteración del algoritmo.

Iter.	Vértice u	Acción / Relajación	A	B	C	D	E	F
$k = 0$	Inicial	Inserción en cola	0 / -	∞ / -	∞ / -	∞ / -	∞ / -	∞ / -
$k = 1$	A ($d = 0$)	Relaja B (180), Relaja C (450)	0 / -	180 / A	450 / A	∞ / -	∞ / -	∞ / -
$k = 2$	B ($d = 180$)	Relaja D (400), Descarta C	0 / -	180 / A	450 / A	400 / B	∞ / -	∞ / -
$k = 3$	D ($d = 400$)	Relaja E (650), Descarta C	0 / -	180 / A	450 / A	400 / B	650 / D	∞ / -
$k = 4$	C ($d = 450$)	Relaja F (800), Descarta E	0 / -	180 / A	450 / A	400 / B	650 / D	800 / C
$k = 5$	E ($d = 650$)	Descarta F	0 / -	180 / A	450 / A	400 / B	650 / D	800 / C
$k = 6$	F ($d = 800$)	Sin aristas salientes	0 / -	180 / A	450 / A	400 / B	650 / D	800 / C

Notación *distancia / predecesor* ($d[v] / \pi[v]$). Cifras en negrita indican nodos cerrados en S .

2.6.1. Reconstrucción de Rutas Mínimas Óptimas

A partir del vector de predecesores $\pi = \{A : \text{NIL}, B : A, C : A, D : B, E : D, F : C\}$, las rutas mínimas desde la Plaza Mayor (A) se reconstruyen mediante retroceso encadenado:

- **Ruta a Mercado San Pedro (E):** $E \leftarrow \pi[E] = D \leftarrow \pi[D] = B \leftarrow \pi[B] = A \implies A \rightarrow B \rightarrow D \rightarrow E$ con costo total de **650.0** m.
- **Ruta a Plazoleta Limacpampa (F):** $F \leftarrow \pi[F] = C \leftarrow \pi[C] = A \implies A \rightarrow C \rightarrow F$ con costo total de **800.0** m.

Ambos resultados analíticos concuerdan de forma idéntica con las salidas producidas por el solver de software implementado en Python.

3. Diseño Algorítmico y Estructuras de Colas de Prioridad

3.1. Algoritmo de Dijkstra con Cola de Prioridad Abstracta

El algoritmo de Dijkstra calcula las rutas mínimas sobre grafos con pesos no negativos operando sobre una cola de prioridad genérica Q . El procedimiento formal estructurado se describe en el Algoritmo 1.

Algorithm 1 Algoritmo de Dijkstra con Cola de Prioridad Parametrizada

Require: Grafo dirigido $G = (V, E, W)$, Vértice origen $s \in V$, Estructura de cola Q

Ensure: Distancias mínimas $d : V \rightarrow \mathbb{R}^+ \cup \{\infty\}$, Predecesores $\pi : V \rightarrow V \cup \{\text{NIL}\}$

```

1: Inicialización:
2: for all  $v \in V$  do
3:    $d[v] \leftarrow \infty$ 
4:    $\pi[v] \leftarrow \text{NIL}$ 
5: end for
6:  $d[s] \leftarrow 0.0$ 
7:  $Q.\text{BUILD}(V, d)$  {O inserción iterativa de cada  $v \in V$  con clave  $d[v]$ }
8:  $S \leftarrow \emptyset$ 
9: while not  $Q.\text{IS\_EMPTY}()$  do
10:   $(u, d_u) \leftarrow Q.\text{EXTRACT\_MIN}()$ 
11:  if  $d_u = \infty$  then
12:    break {Vértices restantes inalcanzables desde  $s$ }
13:  end if
14:   $S \leftarrow S \cup \{u\}$ 
15:  for all  $(u, v) \in E$  do
16:    if  $v \notin S$  then
17:       $\text{candidate\_dist} \leftarrow d_u + W(u, v)$ 
18:      if  $\text{candidate\_dist} < d[v]$  then
19:         $d[v] \leftarrow \text{candidate\_dist}$ 
20:         $\pi[v] \leftarrow u$ 
21:         $Q.\text{DECREASE\_KEY}(v, d[v])$  {Actualización de prioridad en  $Q$ }
22:      end if
23:    end if
24:  end for
25: end while
26: return  $(d, \pi)$ 

```

3.2. Invariantes de Bucle y Prueba de Corrección

La corrección del algoritmo descansa en el siguiente invariante formal de bucle:

Invariante: Al inicio de cada iteración del bucle `while` (línea 9), para todo vértice $u \in S$, el valor $d[u]$ es exactamente la distancia mínima desde el origen s a u , esto es, $d[u] = \delta(s, u)$. Además, para cada vértice tentativo $v \in Q = V \setminus S$, el valor $d[v]$ representa la longitud del camino más corto desde s hasta v cuyos vértices intermedios pertenecen exclusivamente al conjunto de permanentes S .

Demostración por Inducción Matemática:

- **Base** ($|S| = 0$): Antes de la primera iteración, $S = \emptyset$. Para el origen s , $d[s] = 0 = \delta(s, s)$ puesto que los pesos son no negativos ($w(e) \geq 0$). Para todo $v \neq s$, $d[v] = \infty$, lo cual satisface la cota superior vacía.
- **Paso Inductivo:** Supóngase que el invariante se cumple para un conjunto S de tamaño k . El algoritmo selecciona $u \in Q$ tal que $d[u] = \min_{z \in Q} d[z]$. Si existiese un camino más corto P desde s a u , dicho camino debe abandonar S por primera vez a través de una arista (x, y) con $x \in S$ y $y \in Q$. La longitud de este camino sería:

$$w(P) \geq \delta(s, x) + w(x, y) = d[y] \geq d[u] \quad (12)$$

dado que los pesos son no negativos y u minimiza la distancia tentativa en Q . Por lo tanto, no puede existir un camino más corto que $d[u]$, implicando $d[u] = \delta(s, u)$.

- **Terminación:** Cuando Q queda vacía o los nodos restantes tienen distancia ∞ , $S = V$ (para la componente conexa alcanzable), garantizando que $d[v] = \delta(s, v)$ para todo $v \in V$.

3.3. Estructuras de Datos de Cola de Prioridad Evaluadas

3.3.1. Min-Heap Binario Indexado (IndexedMinHeap)

En un montículo binario estándar implementado sobre un arreglo contiguo, la operación `Decrease-Key(v, nueva_clave)` requiere localizar previamente el nodo v , lo cual demanda una búsqueda lineal $O(|V|)$ inaceptable para grafos grandes.

Nuestra implementación desacoplada resuelve este cuello de botella mediante una tabla hash inversa de posiciones $\text{pos} : V \rightarrow \mathbb{Z}^+$. Cuando dos elementos en las posiciones i y j del arreglo se intercambian durante las rutinas de flotado (`_sift_up`) o hundido (`_sift_down`), sus entradas en pos se actualizan de forma atómica en $O(1)$:

$$\text{heap}[i], \text{heap}[j] \leftarrow \text{heap}[j], \text{heap}[i]; \quad \text{pos}[\text{heap}[i].\text{id}] \leftarrow i; \quad \text{pos}[\text{heap}[j].\text{id}] \leftarrow j \quad (13)$$

Con este mecanismo, `Decrease-Key` ubica el índice en $O(1)$ y ejecuta a lo sumo $\lfloor \log_2 |V| \rfloor$ intercambios hacia la raíz, logrando una complejidad estricta de $O(\log |V|)$.

3.3.2. 4-ary Heap Contiguo (DAryHeap)

El d -ary Heap generaliza el montículo binario permitiendo que cada nodo posea d hijos. Para un índice de nodo i (indexación base cero):

$$\text{padre}(i) = \left\lfloor \frac{i-1}{d} \right\rfloor, \quad \text{hijo}_k(i) = d \cdot i + k \quad (\text{con } 1 \leq k \leq d) \quad (14)$$

Al fijar $d = 4$, la altura máxima del árbol se reduce a $\lceil \log_4 |V| \rceil = \frac{1}{2} \lceil \log_2 |V| \rceil$. Aunque la operación `_sift_down` debe inspeccionar 4 hijos para hallar el menor (requiriendo 3 comparaciones por nivel), la operación crítica `Decrease-Key` ejecuta el flotado ascendente `_sift_up` con una única comparación por nivel:

$$T_{\text{Decrease-Key}} = O(\log_4 |V|) = O\left(\frac{\log_2 |V|}{2}\right) \quad (15)$$

Adicionalmente, los 4 hijos de cualquier nodo residen en posiciones contiguas del arreglo, lo que maximiza la probabilidad de que se alojen dentro de la misma línea de memoria caché L1/L2 (típicamente de 64 bytes), reduciendo drásticamente los fallos de caché frente a árboles dinámicos con punteros dispersos (LaMarca & Ladner, 1996).

3.3.3. Fibonacci Heap (FibonacciHeap)

El Fibonacci Heap (Fredman & Tarjan, 1987) es una colección de árboles de orden min-heap enraizados en una lista circular doblemente enlazada. A diferencia de los montículos basados en arreglos, no impone una forma rígida de árbol completo. Sus propiedades matemáticas distintivas son:

1. **Inserción Perezosa ($O(1)$ peor caso):** Los nuevos nodos se incorporan a la lista de raíces sin consolidar.
2. **Disminución de Clave ($O(1)$ amortizado):** Si al decrecer la clave de x se viola la propiedad de montículo frente a su padre $y = p[x]$, el subárbol enraizado en x se corta inmediatamente y se transfiere a la lista de raíces. Si el padre y ya había perdido previamente un hijo ($\text{mark}[y] = \text{true}$), se desencadena un corte en cascada (*cascading cut*) recursivo hacia los ancestros.
3. **Extracción del Mínimo ($O(\log |V|)$ amortizado):** Se extrae la raíz mínima y todos sus hijos se promueven a la lista raíz. A continuación, se ejecuta una rutina de consolidación vinculando raíces de igual grado mediante un arreglo auxiliar de punteros de tamaño $O(\log |V|)$.

3.3.4. Pairing Heap (Perspectiva Teórica)

El Pairing Heap es un árbol autoajutable multi-camino que simplifica radicalmente la estructura del Fibonacci Heap al no requerir el almacenamiento de grados ni banderas de corte, logrando tiempos empíricos comparables y $O(\log |V|)$ amortizado para `Decrease-Key`.

3.4. Matriz Comparativa de Complejidad Asintótica

La Tabla 3 resume formalmente las cotas asintóticas temporales y el costo total de ejecución del Algoritmo de Dijkstra según la cola de prioridad empleada.

Tabla 3

Matriz comparativa de complejidades temporales y espaciales de colas de prioridad en Dijkstra

Estructura	Insert	Extract-Min	Dec-Key	Espacio	Costo Total Dijkstra	Régimen Óptimo en Red Vial
Arreglo No Ord.	$O(1)$	$O(V)$	$O(1)$	$O(V)$	$O(V ^2)$	Grafos densos ($ E \approx V ^2$).
Binary Min-Heap	$O(\log V)$	$O(\log V)$	$O(\log V)$	$O(V)$	$O((V + E) \log V)$	Grafos dispersos estándar.
Montículo 4-ario	$O(\log_4 V)$	$O(4 \log_4 V)$	$O(\log_4 V)$	$O(V)$	$O((4 V + E) \log_4 V)$	Alta localidad en línea de caché.
Fibonacci Heap	$O(1)$	$O(\log V)^*$	$O(1)^*$	$O(V)$	$O(E + V \log V)$	Óptimo teórico ($ E \gg V $).
Pairing Heap	$O(1)$	$O(\log V)^*$	$o(\log V)^*$	$O(V)$	$O(E + V \log V)$	Menor sobrecarga de enlaces.

Nota. El asterisco (*) denota costo amortizado según el método del potencial. Para montículos d -arios ($d = 4$), la cota es $O((d \cdot |V| + |E|) \log_d |V|)$.

4. Diseño e Implementación del Sistema

4.1. Arquitectura Modular del Software

El sistema computacional ha sido concebido bajo una arquitectura en capas estrictamente desacoplada, orientada a objetos y funciones puras, garantizando que el núcleo de cálculo de caminos mínimos sea completamente independiente de la representación visual, el almacenamiento en disco o la estructura concreta de cola de prioridad empleada (Figura 2).

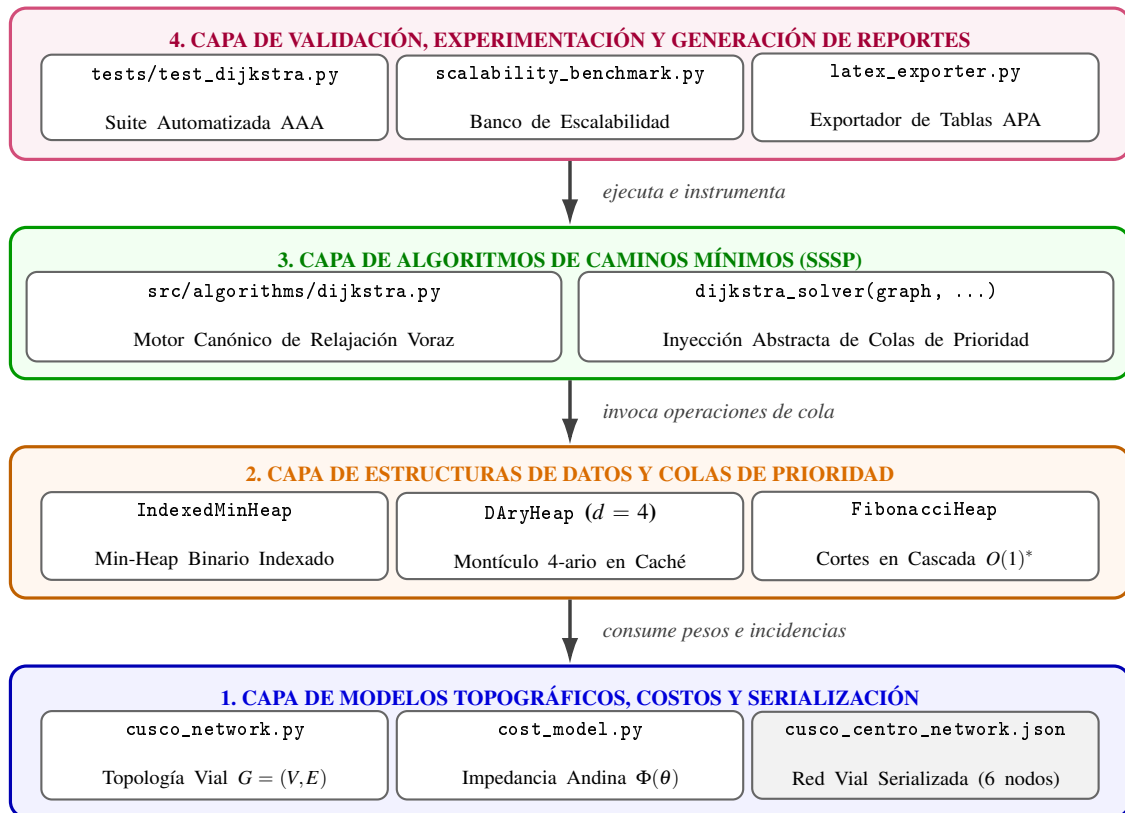


Figura 2

Diagrama arquitectural desacoplado del sistema en cuatro capas operativas

4.2. Descripción de Módulos del Sistema

El repositorio organiza su código fuente en el paquete `src/`:

1. `src/algorithms/dijkstra.py`: Implementa el solver de caminos mínimos con inyección de dependencias. Recibe la lista de adyacencia y una cadena identificadora del tipo de cola (`binary_heap`, `4ary_heap`, `fibonacci_heap`), ejecutando las relajaciones y capturando las estadísticas de operaciones para su posterior análisis.
2. `src/datastructures/indexed_min_heap.py`: Implementación pura del Min-Heap binario con tabla de posiciones invertida para garantizar `decrease_key` en tiempo $O(\log |V|)$.
3. `src/datastructures/d_ary_heap.py`: Implementación del montículo d -ario parametrizado para cualquier aridad $d \geq 2$, configurado por defecto en $d = 4$.

4. `src/datastructures/fibonacci_heap.py`: Implementación del Fibonacci Heap con nodos enlazados circularmente, cálculo de rangos y corte en cascada.
5. `src/models/cusco_network.py`: Mapeo explícito de la topología vial del Centro Histórico de Cusco, serialización y carga en formato JSON.
6. `src/utils/cost_model.py`: Lógica de cálculo del costo generalizado e impedancia por gradiente altitudinal andino.
7. `src/utils/latex_exporter.py`: Rutina automatizada que formatea diccionarios de traza en tablas LaTeX bajo el estándar booktabs.
8. `src/visualization/generate_figures.py`: Renderizador de gráficos vectoriales y matriciales de alta resolución (300 DPI) con Matplotlib y NetworkX.

4.3. Formatos de Datos y Serialización JSON

Para garantizar que la ingestión de datos no dependa de estructuras en memoria volátiles ni rutas locales rígidas, la red vial del Cusco se serializa formalmente en formato JSON bajo la ruta estándar `data/processed/cusco_centro_network.json`:

```
1 {  
2     "nodes": [  
3         {"id": "A", "name": "Plaza Mayor del Cusco"},  
4         {"id": "B", "name": "Plaza Regocijo (Cusipata)"}  
5     ],  
6     "edges": [  
7         {  
8             "origin": "A",  
9             "destination": "B",  
10            "length_m": 180.0,  
11            "street_type": "calle_centro",  
12            "gradient_pct": 1.5,  
13            "street_name": "Calle Espaderos / Portal Carnicerias",  
14            "travel_time_sec": 44.75  
15        }  
16     ]  
17 }
```

Listing 1: Extracto del esquema JSON para la red vial del Centro Histórico de Cusco

4.4. Contratos de Interfaz de las Estructuras de Datos

Para posibilitar la sustitución dinámica y transparente de colas en el solver de Dijkstra sin alterar el flujo algorítmico, cada una de las tres estructuras implementa una interfaz uniforme:

- `insert(item: str, key: float) ->None`: Inserta un identificador de vértice con su distancia tentativa asociada.

- `extract_min()` -> Tuple[str, float]: Extrae y retorna el elemento con menor clave acumulada, reestructurando el montículo según sus invariantes.
- `decrease_key(item: str, new_key: float)` -> None: Disminuye la clave del vértice a su nuevo valor relajado, flotando el nodo o ejecutando cortes en cascada.
- `is_empty()` -> bool: Determina en $O(1)$ si existen vértices pendientes de procesamiento.

4.5. Implementación de Algoritmos y Estructuras Nucleares

A continuación, se presentan los códigos fuente de los algoritmos y estructuras nucleares desarrollados, embebidos directamente desde el repositorio:

```

1  """
2  Algoritmo de Dijkstra con colas de prioridad intercambiables.
3  Genera trazas paso a paso y estadísticas de operaciones.
4  """
5  from typing import Dict, List, Tuple, Any, Optional
6  import heapq
7
8  # Intento de importación de colas especializadas (se integrarán en commits 2 y 3)
9  try:
10     from src.datastructures.indexed_min_heap import IndexedMinHeap
11 except ImportError:
12     IndexedMinHeap = None
13
14 try:
15     from src.datastructures.d_ary_heap import DAryHeap
16 except ImportError:
17     DAryHeap = None
18
19 try:
20     from src.datastructures.fibonacci_heap import FibonacciHeap
21 except ImportError:
22     FibonacciHeap = None
23
24
25 class _FallbackBinaryHeap:
26     """Implementación base de cola de prioridad binaria para el prototipo inicial."""
27     def __init__(self):
28         self._heap = []
29         self._entry_map = {}
30         self._counter = 0
31
32     def is_empty(self) -> bool:
33         return len(self._entry_map) == 0
34
35     def insert(self, node: str, dist: float) -> None:
36         self.decrease_key(node, dist)
37
38     def decrease_key(self, node: str, dist: float) -> None:
39         self._counter += 1
40         entry = [dist, self._counter, node]

```

```

41     self._entry_map[node] = entry
42     heapq.heappush(self._heap, entry)
43
44     def extract_min(self) -> Tuple[float, str]:
45         while self._heap:
46             dist, count, node = heapq.heappop(self._heap)
47             if node in self._entry_map and self._entry_map[node][1] == count:
48                 del self._entry_map[node]
49                 return dist, node
50             raise IndexError("Extracción sobre montículo vacío.")
51
52
53 def dijkstra_solver(
54     nodes: List[str],
55     adj: Dict[str, List[Tuple[str, float]]],
56     source: str,
57     queue_type: str = "binary_heap"
58 ) -> Dict[str, Any]:
59     """Ejecuta Dijkstra con la cola de prioridad indicada. Retorna distancias, caminos
60     , traza y estadísticas."""
61     dist: Dict[str, float] = {v: float('inf') for v in nodes}
62     prev: Dict[str, Optional[str]] = {v: None for v in nodes}
63     dist[source] = 0.0
64
65     # Selección de estructura de cola de prioridad
66     if queue_type == "binary_heap":
67         if IndexedMinHeap is not None:
68             heap = IndexedMinHeap()
69         else:
70             heap = _FallbackBinaryHeap()
71     elif queue_type == "4ary_heap":
72         if DAryHeap is None:
73             raise NotImplementedError("DAryHeap no disponible en esta fase. Pendiente
74             de integración por Yeni Porroa.")
75         heap = DAryHeap(d=4)
76     elif queue_type == "fibonacci_heap":
77         if FibonacciHeap is None:

```

Listing 2: Implementación desacoplada del algoritmo de Dijkstra con inyección de colas (python/src/algorithms/dijkstra.py)

```

1  """
2  Min-Heap Binario con tabla de posiciones para Decrease-Key en O(log V).
3  """
4  from typing import List, Tuple, Dict, Any, Optional
5
6  class IndexedMinHeap:
7      """Cola de prioridad Min-Heap con indexación directa de nodos."""
8      def __init__(self):
9          # heap almacena tuplas (clave_distancia, id_nodo)
10         self.heap: List[Tuple[float, Any]] = []
11         # pos mapea id_nodo -> índice en self.heap
12         self.pos: Dict[Any, int] = {}
13         # Contadores de operaciones elementales para análisis empírico

```

```

14     self.comparisons: int = 0
15     self.swaps: int = 0
16
17     def is_empty(self) -> bool:
18         return len(self.heap) == 0
19
20     def __len__(self) -> int:
21         return len(self.heap)
22
23     def contains(self, node: Any) -> bool:
24         return node in self.pos
25
26     def insert(self, node: Any, dist: float) -> None:
27         idx = len(self.heap)
28         self.heap.append((dist, node))
29         self.pos[node] = idx
30         self._sift_up(idx)
31
32     def extract_min(self) -> Tuple[float, Any]:
33         if not self.heap:
34             raise IndexError("Extracción sobre montículo vacío.")
35         min_dist, min_node = self.heap[0]
36         last_dist, last_node = self.heap.pop()
37         del self.pos[min_node]
38         if self.heap:
39             self.heap[0] = (last_dist, last_node)
40             self.pos[last_node] = 0
41             self._sift_down(0)
42         return min_dist, min_node
43
44     def decrease_key(self, node: Any, new_dist: float) -> None:
45         if node not in self.pos:
46             return
47         idx = self.pos[node]
48         current_dist, _ = self.heap[idx]
49         self.comparisons += 1
50         if new_dist < current_dist:
51             self.heap[idx] = (new_dist, node)
52             self._sift_up(idx)
53
54     def _sift_up(self, idx: int) -> None:
55         while idx > 0:
56             parent = (idx - 1) // 2
57             self.comparisons += 1
58             if self.heap[idx][0] < self.heap[parent][0]:
59                 self._swap(idx, parent)
60                 idx = parent

```

Listing 3: Implementación del Min-Heap Binario Indexado (python/src/datastructures/indexed_min_heap.py)

```

1 """
2 Modelo de costo para ruteo urbano en Cusco. Calcula impedancia considerando
3 longitud, pendiente topográfica (alpha=2.5) y tipo de vía.

```

```

4  """
5  from dataclasses import dataclass
6  from typing import Dict, Tuple
7
8  @dataclass
9  class RoadSegment:
10     """Representa un tramo vial dirigido con atributos físicos y operacionales de
        Cusco."""
11     origin: str
12     destination: str
13     length_m: float          # Longitud métrica en metros
14     street_type: str         # 'avenida', 'calle_centro', 'calle_residencial', '
        peatonal'
15     gradient_pct: float      # Pendiente en porcentaje (e.g., 8.0 para 8% de subida)
16     street_name: str         # Nombre de la vía cusqueña
17
18  class CuscoCostModel:
19     """
20     C(u,v) = T_base * (1 + alpha * max(0, pendiente/100))
21     T_base = longitud / velocidad_nominal
22     """
23     # Velocidades promedio efectivas según tipo de vía en km/h
24     NOMINAL_SPEED_KMH = {
25         'avenida': 35.0,          # Av. El Sol, Av. Tullumayo, Av. de la Cultura
26         'calle_residencial': 25.0, # Calles fuera del casco monumental
27         'calle_centro': 15.0,     # Calles coloniales estrechas (Santa Clara, Marqué
        s, Espaderos)
28         'peatonal': 8.0           # Tramos como Portal de Panes o Loreto con paso
        restringido
29     }
30
31     ALPHA_SLOPE = 2.5 # Sensibilidad a la pendiente (fuerte penalización para vehí
        culos de emergencia en subidas)
32
33     @classmethod
34     def calculate_cost(cls, segment: RoadSegment, metric: str = 'time') -> float:
35         """
36         Calcula el costo del tramo según la métrica elegida:
37         - 'distance': Retorna la distancia física pura en metros.
38         - 'time': Retorna el tiempo de viaje estimado en segundos (impedancia de
        emergencia).
39         """
40         if metric == 'distance':
41             return segment.length_m
42
43         speed_kmh = cls.NOMINAL_SPEED_KMH.get(segment.street_type, 20.0)
44         speed_ms = speed_kmh / 3.6 # Conversión a m/s

```

Listing 4: Modelo de costo e impedancia topográfica andina (python/src/utils/cost_model.py)

Referencias Bibliográficas

- Cherkassky, B. V., Goldberg, A. V., & Radzik, T. (1996). Shortest paths algorithms: Theory and experimental evaluation. *Mathematical Programming*, 73(2), 129-174. <https://doi.org/10.1007/BF02592101>
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269-271. <https://doi.org/10.1007/BF01386390>
- Fredman, M. L., & Tarjan, R. E. (1987). Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3), 596-615. <https://doi.org/10.1145/28869.28874>
- LaMarca, A., & Ladner, R. E. (1996). The influence of caches on the performance of heaps. *ACM Journal of Experimental Algorithmics*, 1, 4-es. <https://doi.org/10.1145/235291.235295>